Master Engineering Systems

Major Project Report: Progress

Developing a Scalable Computer-Vision Pipeline for Human–Interaction Element Detection in Hospital Environments

Pavel Petrovski

HAN University of Applied Sciences

Supervisors: Siddharth Ajaykumar, Jan Benders, Ben Pyman

March 19, 2026

HAN_UNIVERSITY OF APPLIED SCIENCES

Master Engineering Systems – Major Project

Cyber-Physical Systems

HAN University of Applied Sciences

Arnhem

Preface

This report presents the work carried out as part of the Master thesis project in Cyber-Physical Systems at HAN University of Applied Sciences. The project was conducted in collaboration with Ambyon and HAN Automotive Research.

The objective of the thesis is to design and evaluate a scalable computer-vision pipeline for interaction-point perception in hospital environments. All work described in this report was performed by the author.

Disclaimer

This Project Report was written independently by the author as part of the Master of Engineering Systems program at HAN University of Applied Sciences.

During the preparation of this report, the generative artificial intelligence tool *ChatGPT* (OpenAI, 2025) was used selectively to support the writing process. The tool was employed to assist with improving sentence structure, refining academic language, and suggesting minor text rephrasing.

All technical content, system design decisions, experimental implementation, training procedures, evaluation methods, and results presented in this report are the original work of the author. The use of the AI tool did not replace independent analysis, interpretation of scientific literature, or critical reasoning.

The use of AI complies with the *MES Policy on the Use of Generative AI*, ensuring informed, transparent, ethical, and responsible application.

Summary

This project focuses on the design and evaluation of a computer-vision pipeline for detecting and interpreting interaction points, such as elevator buttons, in hospital environments. The work is motivated by the need for robust perception capabilities in service robots operating across different hospital infrastructures.

A scalable workflow is proposed that combines instance segmentation, semi-automatic annotation, and iterative model retraining. A custom dataset is created by extending existing data with instance-level annotations using a CVAT-based annotation pipeline. Multiple instance segmentation models are trained using transfer learning, evaluated using standard and task-oriented metrics, and prepared for deployment on embedded hardware.

The project demonstrates a complete perception pipeline, from dataset creation to embedded deployment, and provides a foundation for future extensions toward fully autonomous robot interaction in clinical environments.

Acronyms and Abbreviations

AI Artificial Intelligence.

AP Average Precision.

APA American Psychological Association.

CNN Convolutional Neural Network.

COCO Common Objects in Context.

CPU Central Processing Unit.

CV Computer Vision.

CVAT Computer Vision Annotation Tool.

DL Deep Learning.

FP16 Half-precision floating point format.

FPS Frames Per Second.

GPU Graphics Processing Unit.

IEEE Institute of Electrical and Electronics Engineers.

IoU Intersection over Union.

mAP Mean Average Precision.

Mask R-CNN Mask Region-Based Convolutional Neural Network.

ML Machine Learning.

NVIDIA Jetson Embedded GPU platform for edge AI.

OCR Optical Character Recognition.

ONNX Open Neural Network Exchange.

RGB Red–Green–Blue.

RGB-D Red–Green–Blue plus Depth.

RNN Recurrent Neural Network.

ROS Robot Operating System.

ROS 2 Robot Operating System 2.

SLAM Simultaneous Localization and Mapping.

SOTA State of the Art.

TBD To Be Determined.

TensorRT TensorRT.

WSL2 Windows Subsystem for Linux 2.

XML Extensible Markup Language.

YOLO You Only Look Once.

Contents

1 Introduction

This introduction includes the background information, problem definition, project objective and the research questions.

1.1 Background

Robotics in healthcare is moving from experimental pilots to practical deployment, supporting tasks such as logistics, delivery of medical supplies, and environmental assistance. These systems are intended not to replace clinical staff, but to support them by reducing repetitive physical work and enabling more time for patient care [6, 14]. As hospitals face increasing operational pressure and staff shortages, autonomous robots capable of safe navigation and interaction with the environment hold significant potential.

Hospitals are typically multi-floor environments that rely on infrastructure such as elevators, automatic doors, and access-control systems. For a robot to move freely and perform tasks across different floors, it must be able to detect and interact with these human–machine interfaces, including elevator buttons and door panels. Reliable perception of such interaction elements is therefore a key capability for achieving truly autonomous navigation in hospital settings. Without this, robots remain confined to limited single-floor operation or require human assistance for vertical transport.

Figure 1.1: CareBot prototype at HAN Automotive Research

At *HAN Automotive Research*, the CareBot platform is being developed as an educational and research project that allows students and researchers to experiment with robotic perception, planning, and control in realistic environments. While not designed to replicate the Ambyon ONE robot, the CareBot in this project will be equipped with a similar sensor stack and processing unit, enabling realistic simulation of Ambyon's operational conditions. The Ambyon ONE itself is a service robot currently under development by *Ambyon*, a Dutch startup focused on robotics for healthcare and logistics applications.

The CareBot system uses an Red–Green–Blue plus Depth (RGB-D) camera and Embedded GPU platform for edge AI (NVIDIA Jetson) computing module to enable on-device perception suitable for real-time operation in clinical environments. A key research challenge is enabling the robot to accurately detect and classify interaction points such as elevator buttons, access readers, and door panels under varying lighting, reflections, occlusions, and diverse panel designs - while maintaining efficient performance within embedded hardware constraints.

This project contributes to that research direction by designing and evaluating a scalable computer-vision workflow for interaction-point detection on embedded systems. The work includes dataset creation, model design and optimisation, evaluation on prototype hardware, and preparation for future extensions. The resulting system will serve as a foundation for autonomous hospital mobility and support the development of assistive robotic technologies.

1.2 Problem Definition

The Ambyon ONE robot currently lacks a reliable computer-vision system to detect and recognize human-interaction elements (e.g., elevator and door control interfaces) in hospital environments, preventing autonomous navigation and interaction with building infrastructure.

1.3 Project Objective

Develop and validate an efficient and scalable computer-vision workflow that enables a service-robot platform to autonomously detect and classify human-interaction elements in hospital environments using embedded hardware.

1.4 Research Question(s)

Main Research Question:
How can an efficient and scalable computer-vision workflow be designed and implemented to enable a service robot to detect and interpret human-interaction elements in hospital environments using embedded hardware?

Sub-questions:

1. What are the relevant human–interaction points in hospital environments that should be detected and classified by the computer-vision workflow?
2. What computer-vision methods and model architectures are most suitable for detecting and classifying human-interaction elements in service-robot environments?
3. How can a high-quality dataset be designed and constructed for achieving robust model performance?
4. Which model-optimization techniques support real-time inference on embedded hardware while preserving detection accuracy?
5. How can the workflow support scalable and efficient model improvements, including potential data collection and retraining during deployment?

1.5 Outline of the report

Chapter 1 introduces the background, motivation, and objectives of the project. Chapter 2 reviews relevant literature on interaction-point perception, instance segmentation, and embedded computer-vision systems. Chapter 3 describes the methodology, including dataset preparation, annotation strategy, model training, evaluation, and deployment considerations. Chapter 4 presents the experimental results and quantitative evaluation. Chapter 5 discusses the findings and their implications, and Chapter 6 concludes the report and outlines future work.

2 Literature Review

This chapter reviews the relevant scientific literature used in the Project, related to interaction-point perception in hospital environments, with a focus on instance segmentation, dataset design, annotation strategies, training and deployment of computer-vision models on embedded systems.

2.1 Perception of human–machine interaction elements

Human–machine interaction elements such as elevator buttons, door panels, and access readers are important targets for indoor service robots operating in hospital environments. Previous work in healthcare and service robotics shows that reliable interaction with such infrastructure is required for autonomous operation across multiple floors and restricted areas [6, 14].

From a computer-vision perspective, these interaction elements are challenging to detect because they are small, visually similar, and often arranged closely together on control panels. Studies on visual perception for robotic interaction report that elevator panels typically contain many buttons with minimal visual differences, which makes robust localization difficult under varying lighting conditions, reflections, and viewing angles [2].

General indoor-robot perception systems often focus on detecting large objects or structural elements and assume that interaction with building infrastructure is externally supported or environment-specific. However, research on autonomous service robots emphasizes that scalable deployment requires perception systems that can directly identify actionable interface elements from camera and sensor data, without relying on prior knowledge of the environment [21].

These findings indicate that interaction-point perception is a fine-grained vision problem in which individual interface elements must be localized accurately within visually dense scenes. This observation motivates the need for vision methods that go beyond coarse detection of panels or regions and instead support precise identification of individual interaction elements.

2.2 Formulating interaction-point perception as a vision problem

Vision-based perception problems in robotics are commonly formulated as image classification, object detection, or semantic segmentation tasks. Each formulation implies different assumptions about the structure of the scene and the required output, which directly affects suitability for robotic interaction.

Image classification assigns a single label to an entire image and is therefore unsuitable for scenarios where multiple interaction elements appear simultaneously. Object detection extends this formulation by predicting bounding boxes and class labels for individual objects, in the case of the project those are elevator buttons. While detection-based approaches have been widely used for indoor perception tasks, bounding boxes provide only coarse spatial localization and do not capture the precise shape of interaction elements [17, 16].

Semantic segmentation addresses this limitation by assigning a class label to each pixel in the image. Formally, semantic segmentation can be expressed as a pixel-wise classification function

$$f_\theta : \mathbb{R}^{H\times W\times C} \to \{1, \ldots, K\}^{H\times W},$$

where H and W denote image height and width, C the number of input channels, and K the number of semantic classes [12]. Although this formulation enables precise localization, all pixels belonging to the same class are treated as a single region. As a result, semantic segmentation cannot distinguish between multiple instances of the same object class.

For interaction-point perception, this limitation is critical. Elevator panels often contain many visually similar buttons, each corresponding to a distinct action. Semantic segmentation assigns the same class label to all pixels belonging to a given category and therefore cannot distinguish between multiple instances of the same object class [12, 5]. As illustrated in Figure 2.1, a semantic segmentation output would label all button pixels identically, without separating individual buttons. This lack of instance-level differentiation makes semantic segmentation unsuitable for downstream decision-making and physical interaction, where each button must be treated as a separate actionable element.

Instance segmentation extends semantic segmentation by jointly performing object detection and pixel-wise classification, producing a separate mask for each object instance. This formulation can be expressed as a set of instance masks

$$\mathcal{M} = \{M_1, M_2, \ldots, M_N\},$$

where each $M_i \in \{0,1\}^{H\times W}$ represents a binary mask for one object instance [5]. Instance segmentation therefore enables both precise spatial localization and differentiation between multiple objects of the same class. Figure 2.1 illustrates the conceptual differences between image classifi-

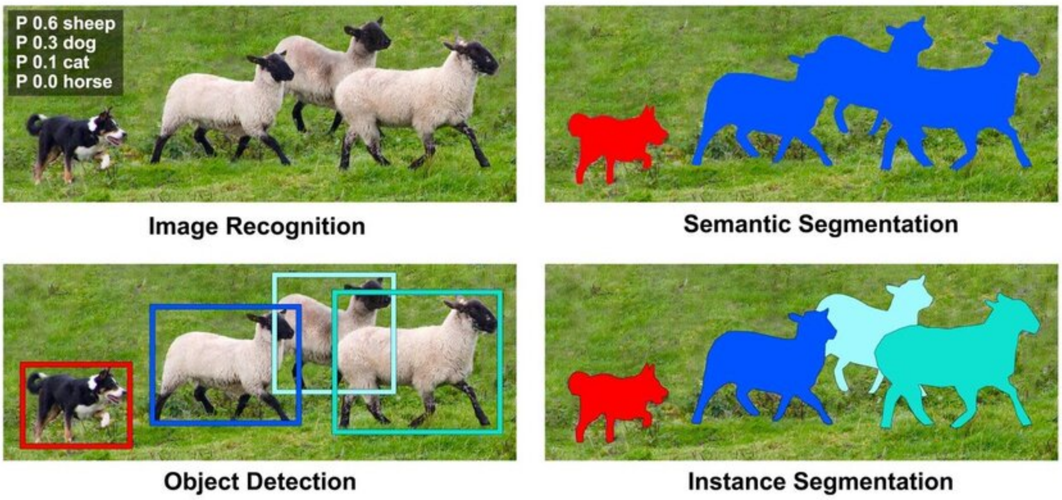

Figure 2.1: Comparison of image classification, object detection, semantic segmentation, and instance segmentation.

cation, object detection, semantic segmentation, and instance segmentation for visual perception tasks.

Based on findings in the literature and the requirements of interaction-point detection, instance segmentation provides the most appropriate problem formulation for this project. It allows individual buttons to be identified as distinct entities while preserving pixel-level accuracy, which is essential for reliable robotic interaction in dense and visually complex control interfaces.

2.3 Public datasets and annotation strategies.

Public datasets for vision-based detection of elevator interfaces are limited, as most indoor robotics datasets focus on navigation or scene understanding rather than interaction elements [19]. This limits their direct applicability to interaction-point perception tasks.

The IRSO 2018 dataset provides Red–Green–Blue (RGB) images of elevator panels captured in realistic indoor environments and was originally introduced for object detection tasks [13]. Due to its focus on real elevator interfaces and visual diversity, it is well suited for studying interaction-point perception in service robotics.

Recent work reports a large-scale dataset with pixel-wise annotations for elevator button segmentation [11]. However, despite being described in the literature, the corresponding semantic segmentation dataset is not publicly available at the time this project is made and therefore cannot be used.

To enable instance-level interaction-point detection, this work reuses images from the IRSO 2018 dataset and replaces the original bounding box annotations with instance segmentation masks. Annotations are stored in the COCO format, which supports instance-level labeling and is widely used across computer vision frameworks, facilitating reuse and future extensions [10].

2.4 Model design principles, evaluation metrics, and semantic interpretation

Instance segmentation models for robotic perception typically follow a modular design in which feature extraction, object localization, and pixel-level mask prediction are handled by separate components within a single network. This design enables shared visual representations while supporting accurate instance-level localization, which is required for interaction-point perception in dense control interfaces.

Region-based instance segmentation models such as Mask Region-Based Convolutional Neural Network (Mask R-CNN) achieve high localization accuracy by explicitly separating region proposal, classification, and mask prediction stages [5]. Due to this design, Mask R-CNN is widely used as a reference architecture and benchmark model in instance segmentation research. However, the multi-stage nature of the model results in high computational cost and memory usage, which limits its suitability for real-time deployment on embedded robotic platforms.

To address these constraints, more lightweight instance segmentation architectures have been proposed that integrate detection and mask prediction into a single-stage pipeline. YOLO-based segmentation models, such as YOLOv8-Seg, trade a moderate reduction in segmentation accuracy for substantially improved inference speed and computational efficiency [8]. These models have demonstrated competitive performance in real-time and embedded settings, making them suitable candidates for deployment on resource-constrained robotic systems.

Model performance is commonly evaluated using overlap-based metrics that measure agreement between predicted masks and ground-truth annotations. Intersection over Union (IoU) is defined as

$$\mathrm{IoU} = \frac{|M_p \cap M_{gt}|}{|M_p \cup M_{gt}|},$$

where M_p and M_{gt} denote the predicted and ground-truth masks. Average Precision (Average Precision (AP)), computed over multiple IoU thresholds, is widely used as a standard evaluation metric for instance segmentation tasks [10].

In many perception pipelines for elevator interfaces, instance localization is followed by a semantic interpretation stage that determines the functional meaning of each detected button. This is commonly achieved using OCR or dedicated symbol classification models applied to the isolated button regions [11]. Accurate instance segmentation is a critical enabling step for such approaches, as it provides clean, well-aligned inputs for subsequent classification or text recognition.

In this work, instance segmentation is treated as the primary perception task, as reliable instance-level localization is a prerequisite for any semantic interpretation of interaction elements. The resulting pipeline is therefore designed to support integration of OCR or custom button classifiers, enabling the robot to associate detected buttons with their intended functions in downstream processing stages.

2.5 Embedded deployment constraints in robotic perception

Vision-based perception systems deployed on mobile service robots must operate under strict computational and energy constraints. Prior work in robotic vision reports that deep-learning models deployed on embedded platforms are limited by onboard memory, processing power, and power consumption, which directly affects real-time performance [3].

Studies evaluating deep-learning inference on NVIDIA Jetson-class hardware show that models achieving high accuracy on desktop Graphics Processing Units (GPUs) often exhibit increased latency and memory usage when deployed on embedded devices [9]. These findings indicate that benchmark accuracy alone is not a sufficient criterion for model selection in robotic perception systems.

For instance segmentation tasks, this trade-off is particularly relevant, as mask prediction introduces additional computational overhead compared to bounding-box detection. Research on efficient model design highlights that architectural choices such as backbone depth and feature reuse have a significant impact on inference efficiency [20]. Based on these insights, this project treats model efficiency and deployability as primary design constraints alongside segmentation accuracy.

Consequently, instance segmentation models considered in this work are evaluated not only with respect to localization performance, but also in terms of their suitability for embedded deployment. This literature-driven perspective directly informs the model selection and optimization strategies described in Chapter 3(Methodology).

2.6 Scalability, robustness, and reproducibility.

Several studies emphasize that robust perception systems require more than a single well-performing model. Instead, scalability is achieved through structured data management, consistent annotation practices, and reproducible training pipelines that support incremental dataset expansion and model updates [4]. These practices reduce sensitivity to domain-specific bias and enable adaptation to new environments without redesigning the entire system.

From these findings, this project adopts a pipeline-oriented approach in which dataset creation, annotation, training, and evaluation are treated as modular and repeatable processes. Rather than optimizing a single model for a fixed dataset, the focus is placed on maintaining a perception workflow that can be extended with new data and retrained as deployment conditions evolve.

2.7 Summary and identified research gap

The reviewed literature shows that reliable perception of interaction elements in robotic systems requires instance-level localization, suitable annotation strategies, and awareness of embedded deployment constraints. Existing studies demonstrate the technical feasibility of these components, but often address them in isolation.

At the same time, publicly available datasets for elevator interfaces are limited, and existing annotations are commonly designed for object detection rather than segmentation tasks. In addition, many approaches focus on model performance without explicitly considering scalability, reproducibility, or deployment on embedded robotic platforms.

This project addresses these gaps by repurposing a realistic elevator dataset with instance-level annotations and by developing a scalable and deployment-aware computer-vision pipeline for interaction-point perception in hospital environments. The following chapter describes the methodology used to realize this approach.

3 Methodology

This chapter describes the methodology used to develop a computer-vision pipeline for interaction-point perception in hospital environments. The project is conducted with the objective of building a scalable and deployment-oriented perception system that can be iteratively improved across different hospital settings and reused within a robot fleet.

3.1 Overview of the proposed perception pipeline

The proposed methodology focuses on instance-level localization of interaction elements such as elevator buttons and control panels. Instance segmentation is used to detect individual button instances at pixel level, allowing precise localization and separation of closely spaced elements. This capability provides the basis for subsequent interpretation of button functions and autonomous interaction.

The methodology is designed to support an iterative training and deployment workflow. An initial base model is deployed on the robot, where it performs interaction-point perception and collects visual data during operation. Selected data can be transferred to an offboard system, where annotations may be reviewed, corrected, or extended before retraining the perception models. The updated models are then redeployed to the robot and become the new default for continued operation.

This workflow enables gradual adaptation to new hospital environments and allows knowledge acquired in one deployment to be reused and refined in subsequent deployments. The target end-to-end training and deployment process toward which this project is developed is illustrated in Figure 3.1.

The following sections describe each stage of the methodology in detail, beginning with dataset selection and preparation.

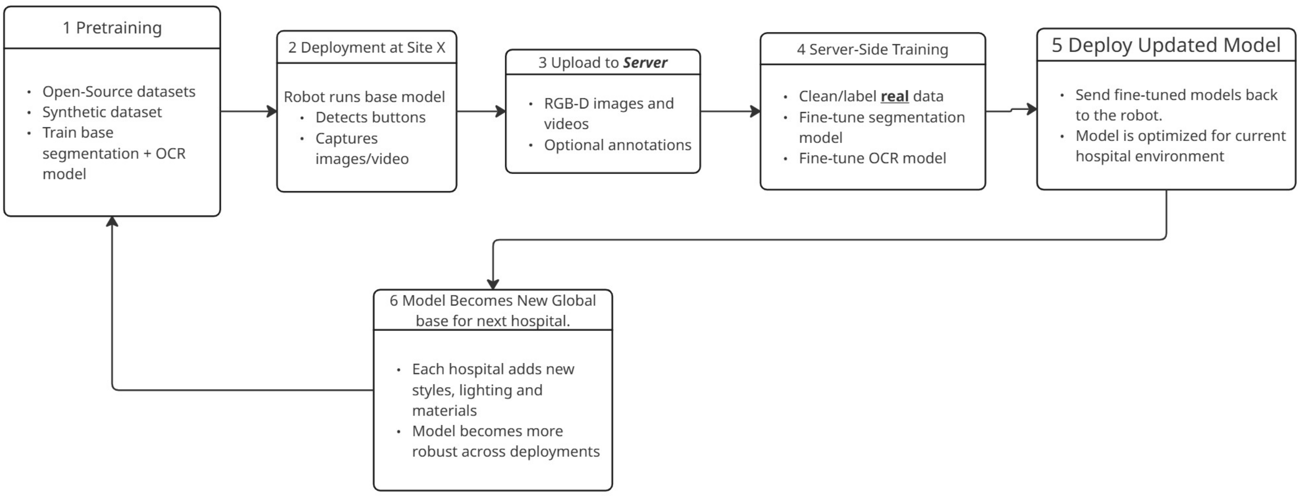

Figure 3.1: Target workflow for scalable training and deployment of interaction-point perception models across multiple hospital environments.

3.2 Dataset selection and preparation

This project uses the elevator-panel dataset released by Liu et al. [11]. For this thesis, the working dataset comprised approximately 2000 images, split into 1122 training images, 281 validation images, and 602 held-out test images. The test split was reserved for final evaluation only.

The original dataset provided Extensible Markup Language (XML) annotations with bounding boxes and textual labels, but no instance masks. To support instance segmentation, the images were re-annotated in a semi-automatic Computer Vision Annotation Tool (CVAT) workflow using a Mask R-CNN pre-annotation model, as described in Section 3.3. The resulting annotations were stored in COCO-style JavaScript Object Notation (JSON) and then converted to the label format required by YOLO-based segmentation training.

In the later deployment-oriented adaptation stage, the dataset was extended with 46 additional images collected in HAN University Building 31, the pilot environment for robot deployment testing. These images were added only after the baseline and hyperparameter tuning stages, so that the initial YOLO experiments remained directly comparable with the original benchmark setting.

Figure 3.2: Example raw image from the dataset.

The resulting dataset structure provides a consistent and reusable foundation for annotation, model training, and evaluation, as described in the following sections.

In addition to the instance-segmentation datasets, a recognition dataset for the semantic interpretation stage was derived from the same annotated source images. For this purpose, each annotated button instance was cropped from the original image using its existing bounding-box annotation, with an additional contextual padding of 15% of the box width and height. This produced a button-centered crop dataset for OCR fine-tuning without requiring any additional manual image collection.

The OCR dataset builder grouped crops by source image before creating a fixed 80/20 train/validation split with a deterministic random seed. As a result, all button crops originating from one panel image were placed either in the training set or in the validation set, but never in both. This prevents information leakage caused by highly similar button appearances within the same elevator panel. The held-out OCR test set was generated directly from the original test images and their annotations.

Several OCR dataset variants were explored during development. The final variant used for the main recognition experiments removed labels named exactly `text` and labels starting with the prefix `text_`. These labels do not encode the actual button content, but merely indicate the presence of unspecified free text. Retaining them caused the recognizer to over-predict generic "TEXT"-like outputs and reduced generalization to unseen textual button content. The final OCR dataset, contained 11,953 training crops, 2,973 validation crops, and 6,861 held-out test crops.

3.3 Annotation strategy and workflow

Instance-level ground truth was created using a semi-automatic workflow that combines model-based pre-annotation with manual verification. This strategy was used to efficiently generate high-quality instance segmentation annotations while maintaining consistency across the dataset.

Annotations were created in CVAT [18], deployed locally using Docker [15] within a Windows Subsystem for Linux 2 (WSL2)-based Linux environment on a Windows host. Automatic annotation was enabled through CVAT–Nuclio integration, where the model was deployed as a Nuclio serverless function [7]. The instance segmentation model used for automatic annotation was Mask R-CNN [5].

During automatic annotation, CVAT sends task images to the Nuclio function, where Mask R-CNN predicts binary masks for each detected button instance. These masks are converted into polygon shapes within the Nuclio function and returned to CVAT as annotation data. The overall workflow used in this project is shown in Figure 3.3.

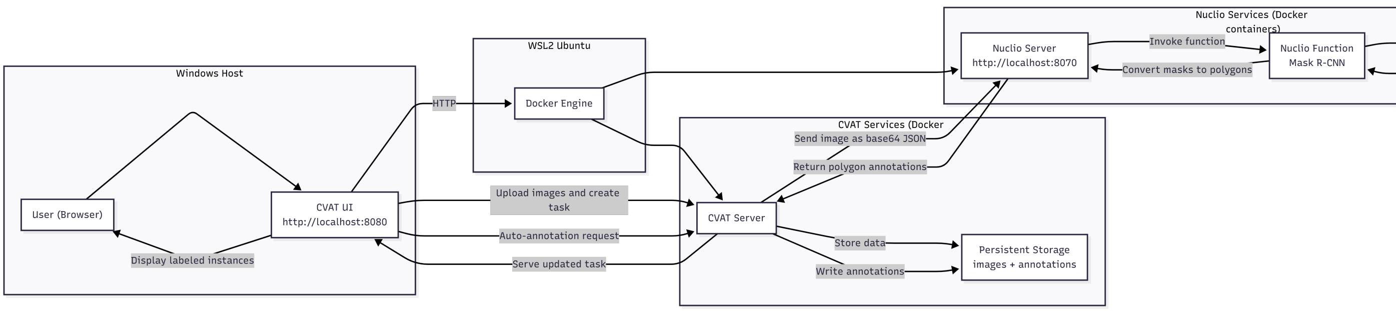

Figure 3.3: Semi-automatic annotation workflow on a Windows host using WSL2 and Docker. CVAT sends images to a Nuclio function running Mask R-CNN, which returns polygon instance annotations derived from predicted binary masks.

To support iterative improvement, the dataset was annotated in batches of approximately 150 images. After each batch, the model was retrained using the newly annotated data and reused for subsequent annotation rounds. Model checkpoints were created after approximately 90, 449, and 900 annotated images, followed by a final checkpoint trained on the full 1400-image training split. This enabled progressively improved pre-annotations and reduced manual correction effort. A quantitative comparison of these annotation-support checkpoints on the held-out test set is provided in Appendix A.

All automatically generated annotations were also manually reviewed and corrected when needed to ensure accurate instance boundaries, correct separation of adjacent buttons, and consistent labeling across both the training and test sets.

Figure 3.4: Example of an annotated image showing instance segmentation masks.

3.4 Dataset quality assessment

A systematic dataset quality assessment was performed to check whether image-level quality issues could bias model learning.

3.4.1 Statistical quality metrics.

For every training image, the following metrics were computed:

- **Sharpness**, estimated using the variance of the Laplacian:

$$\sigma^2_{\text{lap}} = \text{Var}(\nabla^2 I) \tag{3.1}$$

  where I is the grayscale image.

- **Brightness**, defined as the mean grayscale intensity.
- **Edge density**, defined as the fraction of pixels with gradient magnitude above the 90th percentile.

The analysis did not reveal a severe low-quality subset that should be removed. Low-score images were often hard but valid samples, so no global quality-based pruning was applied.

3.4.2 Model-assisted image assessment.

In addition to image-level quality analysis, a model-assisted approach was used to identify potentially harmful training samples.

Method. A trained instance segmentation model was used to evaluate each training image individually. For each image, the following metrics were computed:

- Mean Intersection over Union:

$$\text{IoU} = \frac{|M_{\text{pred}} \cap M_{\text{gt}}|}{|M_{\text{pred}} \cup M_{\text{gt}}|} \tag{3.2}$$

- Mean detection confidence:

$$\bar{c} = \frac{1}{N} \sum_{i=1}^{N} p_i \tag{3.3}$$

  where p_i is the softmax score of the i-th predicted instance.

- Number of false positives and false negatives.

Images were ranked using a composite criterion favoring low IoU, high false positives, and high confidence incorrect predictions. Based on those criteria, a harm-score is measured, based on which training set images are filtered and the potentially harmful ones are selected for manual inspection.

Inspection. Manual inspection of the lowest-ranked samples revealed two categories:

- Hard but valid images (extreme viewpoints, zoom levels, or geometric distortions).
- Negative images without buttons, occasionally producing false positives.

After manual inspection, removing hard but valid examples reduced recall. In contrast, excluding a small number of clear false-positive-only negatives had only marginal effect. The final training policy therefore used the full dataset with minimal exclusions.

3.5 Model architectures and training strategy

Two instance segmentation model families are employed in this project, each serving a distinct role within the overall perception pipeline. Mask R-CNN is used as a high-accuracy reference model and as the backbone of the semi-automatic annotation workflow (Section 3.3), while YOLOv8-Seg is trained and evaluated as a deployment-oriented solution suitable for embedded hardware.

Experimental hardware and software environment

All models training experiments reported in this thesis were executed on the same workstation. The host system ran Ubuntu 22.04.5 LTS with Linux kernel 6.8.0-90. The machine was equipped with an AMD Ryzen 9 9950X 16-core processor, 176 GiB of system memory, and an NVIDIA GeForce RTX 4090 graphics card with 24 GiB of Video Random Access Memory (VRAM). The training software environment used Python 3.10.12 and PyTorch 2.5.1 compiled against CUDA 12.1. This configuration provided the computational platform for the baseline training, hyperparameter tuning, OCR fine-tuning, and the comparative evaluation runs discussed in Chapters 3 and 4.

Common training strategy

All trainable vision models were fine-tuned from pretrained weights rather than trained from scratch. Across the experiments, the train/validation split was used for model fitting and model selection, while the held-out test split was reserved for final comparison only. This separation was especially important in the YOLO experiments, where the best validation model did not always remain the best model on the final test set.

Mask R-CNN training

Mask R-CNN was used only as an annotation-support model within the semi-automatic CVAT workflow. The model was trained with a custom PyTorch-based pipeline on COCO-formatted instance annotations and was retrained iteratively as the annotated dataset grew. Across these retraining stages, the training setup remained based on transfer learning, AdamW optimization, and validation-loss-driven checkpoint selection. The exact checkpoint progression and its held-out test performance are summarized in Appendix A; only the role of Mask R-CNN in the annotation workflow is relevant to the main methodology.

YOLOv8-Seg training

YOLOv8-Seg is trained using the Ultralytics framework with the pretrained `yolov8s-seg.pt` checkpoint. The initial baseline configuration was trained for 100 epochs with an input resolution of 640×640, dynamic batch sizing (`batch=-1`), seed 123, warmup over 5 epochs, and early stopping patience of 10 epochs. The augmentation policy followed the standard Ultralytics recipe used throughout this study, including HSV perturbation, small geometric transformations, horizontal flipping, and mosaic augmentation.

Although this baseline produced strong results, its training settings were selected empirically. A structured hyperparameter tuning procedure was therefore conducted to determine whether performance could be improved in a more systematic and reproducible way. All tuning runs were logged and tracked in Weights & Biases (W&B), which was used to store the hyperparameter configurations, training curves, and validation metrics of individual experiments.

The three stages of the hyperparameter tuning procedure are summarized in Table 3.1. The table is intended to provide a compact overview of the explored search spaces and stopping criteria, while the accompanying text explains the rationale behind each stage.

Table 3.1: Three-stage hyperparameter tuning procedure for YOLOv8-Seg.

Stage	Purpose	Tuned or varied settings	Training setup	Selection rule
Stage 1	Narrow hyperparameter search around the baseline recipe	`lr0`, `lrf`, `weight_decay`, `momentum`, `hsv_h`, `hsv_s`, `hsv_v`, `degrees`, `translate`, `scale`, `mosaic`, `fliplr`, `warmup_epochs`, `close_mosaic`	24 trials, 30 epochs each; sanity run of 12 epochs beforehand	Top runs by validation Mean Average Precision (mAP)
Stage 2	Capacity sweep using the best Stage 1 configurations	Batch size $\in \{8, 16\}$, image size $\in \{640, 768\}$; Stage 1 parameters kept fixed	100 epochs, patience 15	Top 5 Stage 1 runs with $\text{mAP}^{(M)}_{50:95} \geq 0.30$
Stage 3	Late-stage refinement from the best Stage 2 checkpoints	Resume from best checkpoint, halve `lr0`	180 epochs	Top 2 Stage 2 runs with $\text{mAP}^{(M)}_{50:95} \geq 0.50$

The tuning process itself is summarized in Table 3.1. In short, Stage 1 performed a narrow search around the baseline recipe, Stage 2 tested only capacity-related settings such as batch size and image resolution for the best Stage 1 candidates, and Stage 3 resumed the strongest Stage 2 checkpoints for late-stage refinement. All tuning runs were tracked in W&B. After this three-stage tuning procedure, two further experiments were performed: a comparison with YOLO26-small and a domain-adaptation fine-tuning run using the mixed dataset that included images collected in the target deployment environment.

Loss functions and optimization objectives

Both instance segmentation models are trained by minimizing a composite loss function that jointly optimizes object classification, localization, and mask prediction. Although Mask R-CNN and YOLOv8-Seg differ architecturally, their optimization objectives are conceptually aligned.

Mask R-CNN. The training objective of Mask R-CNN is defined as a multi-task loss composed of three terms [5]:

$$\mathcal{L}_{\text{Mask R-CNN}} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{mask}}, \tag{3.4}$$

where $\mathcal{L}_{\text{cls}}$ denotes the classification loss, implemented as categorical cross-entropy; $\mathcal{L}_{\text{box}}$ represents bounding-box regression using the smooth L_1 loss; and $\mathcal{L}_{\text{mask}}$ is a pixel-wise binary cross-entropy loss applied to each predicted instance mask.

YOLOv8-Seg. YOLOv8-Seg integrates detection and segmentation into a single-stage architecture. The total training loss is expressed as a weighted sum of classification, localization, and segmentation losses [8]:

$$\mathcal{L}_{\text{YOLO}} = \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{seg}}\mathcal{L}_{\text{seg}}, \tag{3.5}$$

where $\mathcal{L}_{\text{seg}}$ represents the segmentation loss applied to the predicted mask outputs, and the weighting coefficients λ control the relative contribution of each task during optimization.

Despite architectural differences, both models optimize comparable objectives, enabling consistent evaluation of segmentation accuracy and computational efficiency across reference and deployment-oriented architectures.

OCR-based button interpretation

Following instance segmentation, the detected button regions are passed to a semantic interpretation stage. Segmented button instances are cropped and processed by an OCR-based recognition module to extract character labels or symbols associated with each button. This stage enables the robot to associate detected interaction points with their functional meaning and supports downstream decision-making during autonomous operation.

The semantic interpretation stage is formulated as a cropped-image recognition problem rather than as full-scene text detection. This design choice follows directly from the preceding instance segmentation stage: once a button instance has already been localized, the recognition model no longer needs to solve the harder problem of finding text within the full image. Instead, it receives a tightly bounded crop around a single button and predicts the button's semantic label. This reduces computational cost and makes the OCR stage more suitable for embedded deployment.

Recognition model architecture. For cropped-button recognition, a lightweight OCR recognizer based on `PP-OCRv5_mobile_rec` was selected. This model was preferred because it is designed for efficient deployment and already provides a compact recognition backbone suitable for embedded hardware. In simplified form, the recognizer consists of a compact visual feature extractor, a sequence-modeling stage that arranges features in reading order, and a recognition head that predicts the final label string. In the selected configuration, the feature extractor is based on `PPLCNetV3` with scale 0.95, meaning that a slightly reduced-width version of the backbone is used to keep the model compact. The sequence-modeling stage is `SVTR`-based, and the recognition head combines `CTC` and `NRTR` objectives during training. This architecture is appropriate for the current task because button labels range from single digits and single letters to short strings such as `10`, `4A`, or `CLOSE`, while still requiring efficient inference on cropped inputs.

Only the recognition component of the OCR stack was used. The detection component of Paddle Optical Character Recognition (PaddleOCR) was intentionally omitted from the final recognition pipeline, because button localization was already solved upstream by the instance segmentation model. This avoids redundant computation and makes the OCR stage conceptually cleaner: segmentation localizes the button instance, and OCR interprets that localized instance.

Fine-tuning strategy. The recognizer was initialized from the official pretrained English `PP-OCRv5_mobile_rec` weights and fine-tuned on the project-specific crop dataset. During training, each crop was resized to the fixed input resolution of 48×320 defined in the PaddleOCR recognition configuration so that all samples reached the network with a consistent image height and width. Optimization was performed with Adam, cosine learning-rate scheduling, warmup in the early training phase, and L2 regularization. In this context, cosine learning-rate scheduling means that the step size starts relatively high and then decreases smoothly during training, allowing larger parameter updates in the early epochs and more conservative refinement in the later epochs.

Recognition-specific data augmentation and multiscale sampling were enabled to improve robustness to appearance variation and to reduce overfitting. In practice, this prevents the recognizer from seeing each button crop in only one fixed presentation and improves tolerance to realistic differences in scale, contrast, blur, and local appearance.

The implementation followed the standard PaddleOCR training workflow. Rather than building a custom OCR training loop from scratch, the official `tools/train.py` and `tools/eval.py` scripts provided by the PaddleOCR framework were used directly. Project-specific experiments were defined through YAML Ain't Markup Language (YAML) configuration files and command-line overrides, in the same spirit as using configuration-driven training through Ultralytics for the YOLO experiments.

Model selection followed a two-stage hyperparameter tuning procedure. The first stage varied only the total number of epochs while keeping the baseline optimizer settings fixed. Epoch budgets of 30, 50, 60, and 80 were evaluated in order to determine whether the original

60-epoch recipe was necessary. This stage showed that the model reached its best validation performance before epoch 50 and that training beyond this point produced negligible gain. The second stage therefore fixed the epoch budget at 50 and performed a narrow sweep over optimizer hyperparameters, primarily learning rate and weight decay. The main configurations tested were $\{3 \times 10^{-4}, 5 \times 10^{-4}, 7 \times 10^{-4}\}$ for the initial learning rate and $\{1 \times 10^{-5}, 3 \times 10^{-5}\}$ for weight decay, while warmup was adjusted conservatively between 3 and 5 epochs. All OCR tuning runs were logged in W&B so that training curves, hyperparameter choices, and validation metrics could be compared systematically.

3.6 Evaluation metrics

The quantitative evaluation in this project follows the metrics produced by the training and validation toolchains that were actually used in the experiments. For the YOLO-based instance-segmentation models, the primary metric is mask mAP under the COCO protocol, because the target task is instance segmentation rather than bounding-box detection alone. For completeness, the corresponding box metrics are also reported. The Mask R-CNN annotation-support model was evaluated separately on the same held-out test split, but this comparison is included only as supplementary material in Appendix A, since Mask R-CNN was not selected as the deployment model.

3.6.1 YOLO segmentation metrics under the COCO protocol

The YOLO experiments report the standard Ultralytics/COCO validation and test metrics for both bounding boxes and segmentation masks. The main metric used for model selection is mask mAP@[0.50:0.95], written in the training logs as `metrics/mAP50-95(M)`. This metric averages AP over IoU thresholds from 0.50 to 0.95 in increments of 0.05 and is the most representative single indicator of segmentation quality. In addition, `metrics/mAP50(M)` is reported to show performance at the less strict 0.50 IoU threshold.

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN} \tag{3.6}$$

$$\text{mAP}_{50:95} = \frac{1}{10} \sum_{t \in \{0.50, 0.55, \ldots, 0.95\}} \text{AP}_t \tag{3.7}$$

The complete set of reported YOLO metrics in this report therefore includes mask precision, mask recall, `mAP50(M)`, and `mAP50-95(M)`, together with the corresponding box metrics `precision(B)`, `recall(B)`, `mAP50(B)`, and `mAP50-95(B)`. These additional values are retained because they help explain the behavior of a model beyond a single summary number. For example, precision and recall indicate whether a model tends to miss valid instances or to produce false positives, while the box metrics make it possible to distinguish localization quality from mask quality. However, when ranking segmentation models in this report, mask mAP@[0.50:0.95] remains the primary comparison criterion.

3.6.2 OCR accuracy and normalized edit distance

The OCR recognition models are evaluated using exact-match accuracy and normalized edit distance. Exact-match accuracy is the fraction of button crops for which the predicted label string matches the ground-truth label exactly after the same normalization procedure is applied to both. This metric is strict and is therefore suitable for the final semantic interpretation task, where predicting `4` instead of `14`, or `OPE` instead of `OPEN`, must be counted as an error.

$$\text{Accuracy}_{\text{OCR}} = \frac{N_{\text{correct}}}{N_{\text{total}}} \tag{3.8}$$

where N_{correct} is the number of exactly correct predictions and N_{total} is the total number of evaluated crops.

In addition to exact-match accuracy, normalized edit distance is used to quantify how close an incorrect prediction is to the target string. This metric is derived from the Levenshtein edit distance between prediction and target and is normalized to the interval $[0, 1]$, where 1 indicates a perfect match. Unlike exact-match accuracy, normalized edit distance gives partial credit to near-correct predictions, which is useful when analyzing OCR behavior on short strings and icon labels that are often confused by only one or two characters.

$$\text{NED} = 1 - \frac{1}{N} \sum_{i=1}^{N} \frac{d_{\text{edit}}(\hat{y}_i, y_i)}{\max(|\hat{y}_i|, |y_i|, 1)} \tag{3.9}$$

where $\hat{y}_i$ is the predicted string, y_i is the corresponding ground-truth string, and d_{edit} denotes the Levenshtein edit distance.

During OCR tuning, validation accuracy was treated as the primary model-selection metric, while normalized edit distance was used as a secondary quality indicator. Final OCR benchmarking was then performed on the held-out OCR test split using the official PaddleOCR evaluation script with the same configuration family as used during training.

3.6.3 Evaluation protocol

For the YOLO experiments, validation metrics were used during training and hyperparameter tuning, while the final model comparisons were performed on a held-out test split that was not used for optimization. This distinction is important in the present work, because some tuned configurations achieved better validation performance without surpassing the baseline on the final test set. Accordingly, the results chapter distinguishes explicitly between validation-time model selection and held-out test-set benchmarking.

The Mask R-CNN checkpoint comparison in the appendix follows the same principle: all checkpoints are evaluated on the same held-out test annotations, allowing a fair comparison of the annotation-support model as the training set grew. Final OCR benchmarking was likewise performed on a held-out test split using the official PaddleOCR evaluation script. Together, these procedures form the basis for the quantitative comparisons presented in Chapter 4 (Results).

3.7 Embedded deployment and optimization

To enable real-time execution on embedded hardware, and wrap up step 1 of the workflow strategy in Figure 3.1, trained models are optimized and deployed on a NVIDIA Jetson Orin platform. The deployment workflow is designed to be identical for both the instance segmentation model and the OCR-based button classification model.

Models are first trained in PyTorch, exported to the Open Neural Network Exchange (ONNX) [1] format, and then converted on the NVIDIA Jetson Orin device into optimized TensorRT engines. Engine generation was performed directly on the target hardware to ensure compatibility with the device-specific GPU architecture. Half-precision floating point format (FP16) precision was used to reduce memory footprint and improve inference speed.

In this project, the intermediate ONNX models were not benchmarked as standalone deployment artifacts. Instead, the final TensorRT `.engine` models were validated directly on the target device by running them on a subset of held-out test images and checking that the predictions were numerically sensible and visually consistent with the trained desktop models.

The resulting TensorRT engines are integrated into ROS 2 nodes responsible for perception tasks. At runtime, image data from the RGB-D camera is passed through the ROS 2 pipeline and processed by the TensorRT engines to produce instance segmentation masks and corresponding button labels.

The complete deployment and optimization workflow is illustrated in Figure 3.5. This approach ensures a consistent and efficient deployment pipeline and enables fair evaluation of different model architectures under identical runtime constraints.

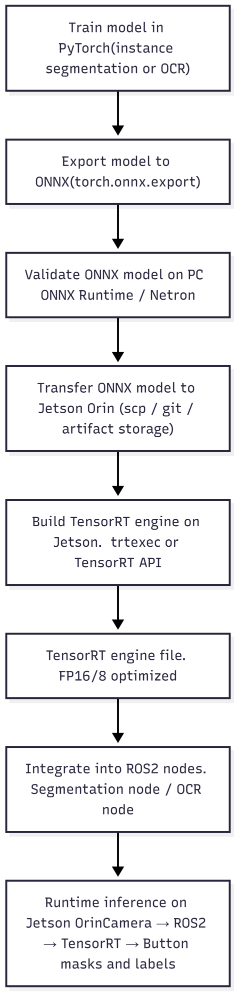

Figure 3.5: Deployment and optimization workflow for models on NVIDIA Jetson Orin using Open Neural Network Exchange (ONNX) and TensorRT (TensorRT).

3.8 ROS2 pipeline architecture

After deployment-oriented model selection, the TensorRT engines were integrated into a modular ROS 2 pipeline on the target robot computer. This stage links the trained perception models to live camera input, downstream message passing, and deployment data collection.

3.8.1 Deployment platform

The runtime pipeline was executed on an NVIDIA Jetson Orin Nano running Ubuntu 22.04.5 LTS and ROS 2 Humble. The device uses NVIDIA Linux for Tegra release `R36.4.7` with Linux kernel `5.15.148-tegra`. TensorRT 10.7.0 was used for optimized inference. Visual input was provided by a Stereolabs ZED X camera connected through a ZED Link Duo capture card, using ZED SDK 5.2.1.

3.8.2 Runtime node graph

The deployed runtime graph consists of the ZED camera driver, a YOLO inference node, an OCR inference node, and an ADC. The ZED wrapper publishes lens-corrected color images, registered depth images, and camera calibration data. The `yolo_engine_node` subscribes to these topics and publishes `/button_targets_3d`. The `ocr_trt_node` subscribes to the camera image topic and `/button_targets_3d`, performs text recognition on cropped button regions, and publishes `/button_targets_3d_ocr`. The `data_collector` then subscribes to the OCR-enriched target stream and the camera image stream for deployment data logging.

The split-node design was chosen deliberately. Earlier experiments showed that placing YOLO and OCR TensorRT inference in a single Python process increased the risk of CUDA-context conflicts and made the system harder to debug. Separating the geometric perception stage from the semantic recognition stage improved modularity and allowed both runtime components to be benchmarked independently.

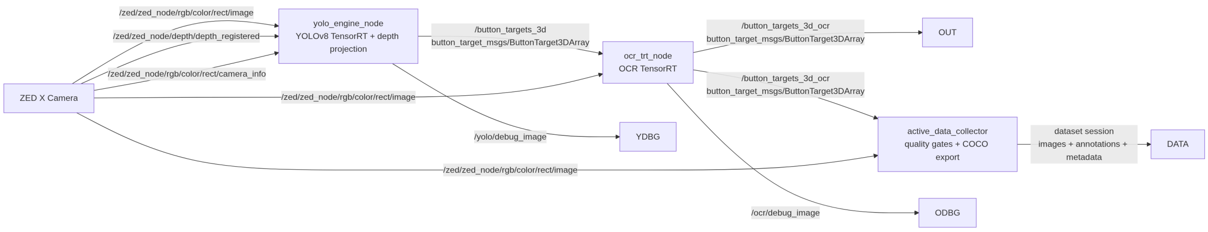

Figure 3.6: ROS 2 runtime architecture used for deployment. The ZED X camera provides image, depth, and camera calibration topics to the YOLO node. The OCR node enriches the geometric targets with text labels, and the active data collector stores accepted deployment samples for later retraining.

Table 3.2: Main ROS 2 nodes in the deployed perception pipeline.

Node	Primary topics	Function
`zed_wrapper`	RGB, depth, camera info	Publishes sensor streams from the ZED X camera.
`yolo_engine_node`	In: RGB, depth, camera info; Out: `/button_targets_3d`	Runs TensorRT segmentation, estimates target depth, and projects detections to 3D.
`ocr_trt_node`	In: RGB, `/button_targets_3d`; Out: `/button_targets_3d_ocr`	Matches detections to buffered images and adds OCR labels.
`data_collector`	In: RGB, `/button_targets_3d_ocr`	Exports selected deployment samples for later review and retraining.

3.8.3 Message flow and collector design

The internal interface between the perception nodes is the custom `ButtonTarget3D` message, which combines 2D detection geometry, 3D position estimates, compact mask data, and OCR fields in a single transport format. Image topics use a best-effort, keep-last quality-of-service setting so that the pipeline always prefers recent sensor data over delayed frames.

Within the `yolo_engine_node`, target depth is estimated only from valid depth pixels that fall inside the predicted segmentation mask. Let $\mathcal{V}$ denote the set of valid masked depth pixels after finite-value filtering, depth range filtering, and the minimum-valid-pixel test. The representative target depth is then computed as

$$z = \text{Percentile}\left(\{d(i,j) \mid (i,j) \in \mathcal{V}\},\, p\right), \tag{3.10}$$

where p is the configured depth percentile. In the current implementation, $p = 50$ is used, corresponding to the median of the valid masked depth samples. Using a percentile-based estimate rather than a plain mean reduces sensitivity to outliers caused by missing returns and reflective elevator surfaces.

The representative image location is also derived from the valid masked depth pixels. In the current implementation, the pixel coordinates are taken as the median valid depth location:

$$u = \text{median}\left(\{j \mid (i,j) \in \mathcal{V}\}\right), \qquad v = \text{median}\left(\{i \mid (i,j) \in \mathcal{V}\}\right). \tag{3.11}$$

If the camera intrinsics are given by focal lengths (f_x, f_y) and principal point (c_x, c_y), the 3D target position in the camera optical frame is computed as

$$x = \frac{(u - c_x)z}{f_x}, \qquad y = \frac{(v - c_y)z}{f_y}. \tag{3.12}$$

This projection step provides the 3D button target used by downstream robotic modules and by the ADC.

To support iterative improvement after deployment, the ADC stores selected images and annotations in a COCO-compatible structure. Sample export is not performed for every frame. Instead, frames pass a sequence of quality gates before they are written to a dataset session. At image level, the collector applies brightness and sharpness checks. Brightness is used to reject severely underexposed or overexposed frames, while sharpness is used to reject blurred frames. At target level, the collector applies minimum score and size thresholds, followed by temporal stability checks and near-duplicate rejection.

An important late-stage refactor moved full-image mask encoding out of the YOLO node and into the collector. In the final design, the YOLO node publishes only a compact ROI-local mask, while the collector reconstructs full-image annotations only for accepted samples. This reduces runtime overhead in the perception path and keeps dataset-generation logic out of the real-time inference loop.

4 Results

This chapter presents the quantitative results of the main perception components of the proposed elevator-interaction pipeline. It first reports the results of the YOLO-based button-instance segmentation system and then summarizes the OCR-based button interpretation results. Unless stated otherwise, final model comparisons are reported on held-out test sets, while model selection during training and hyperparameter tuning is based on the validation split.

4.1 YOLO hyperparameter tuning and evaluation

4.1.1 Baseline YOLOv8-Seg performance

The baseline YOLOv8-Seg model established the main reference point for all subsequent experiments. On the validation split, the best checkpoint was obtained at epoch 49. This model achieved a mask mAP_{50} of 0.93931 and a mask $\mathrm{mAP}_{50:95}$ of 0.59645, with mask precision and recall of 0.90623 and 0.89947, respectively. These results confirmed that the initial configuration was already strong enough to serve as a realistic benchmark rather than only as a preliminary starting point.

When evaluated on the held-out original test split at an image size of 640 pixels, the same baseline model achieved a mask mAP_{50} of 0.91921 and a mask $\mathrm{mAP}_{50:95}$ of 0.58295. This test result is slightly lower than the best validation value, which is expected, but it still indicates good generalization to unseen elevator-panel images.

4.1.2 Hyperparameter tuning results

The three-stage hyperparameter tuning procedure improved validation performance, but the gain was moderate. Stage 1 acted as a narrow search around the baseline recipe. Only a small subset of trials produced stable non-zero mask performance, as shown in Figure 4.1, and only these candidates were carried forward.

Figure 4.2 shows the broader Stage 2 validation mask $\mathrm{mAP}_{50:95}$ trends for the successful unique Stage 2 configurations. Eight out of sixteen unique Stage 2 configurations achieved non-zero mask $\mathrm{mAP}_{50:95}$ values. The strongest Stage 2 result was obtained with batch size 16 and image size 768, which was then forwarded to Stage 3 refinement.

Table 4.1 summarizes the best validation results of the baseline model and the best Stage 3 tuned model. The tuned model improved mask $\mathrm{mAP}_{50:95}$ from 0.59645 to 0.63064 on validation, but this must be interpreted as a model-selection result rather than as final evidence of better generalization.

Figures 4.3 and 4.4 show the two Stage 3 refinement runs. Their trajectories are nearly identical, indicating that the refinement stage was stable once the strongest Stage 2 configuration had been selected.

The held-out test comparison in Table 4.2 does not support replacing the baseline checkpoint with the Stage 3 model. The tuning procedure improved validation performance, but the baseline

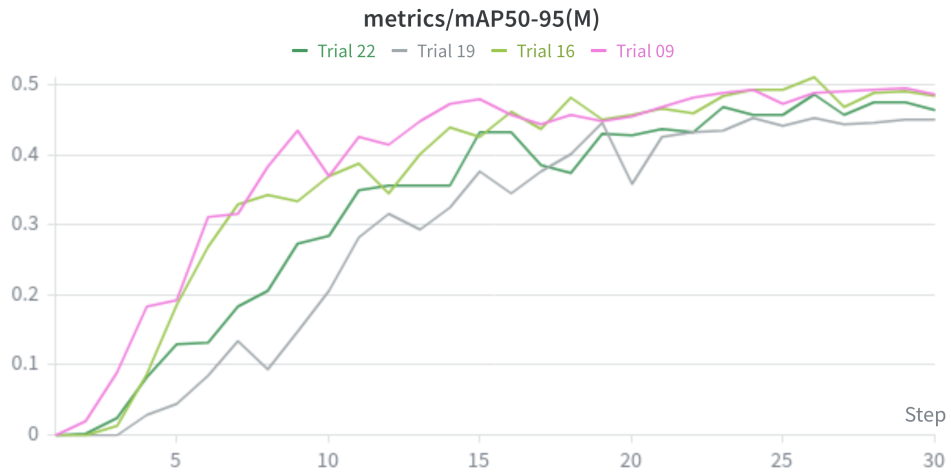

Figure 4.1: Validation mask $\text{mAP}_{50:95}$ for the successful Stage 1 hyperparameter trials. Only a limited subset of the Stage 1 configurations produced stable non-zero segmentation performance and was considered for Stage 2.

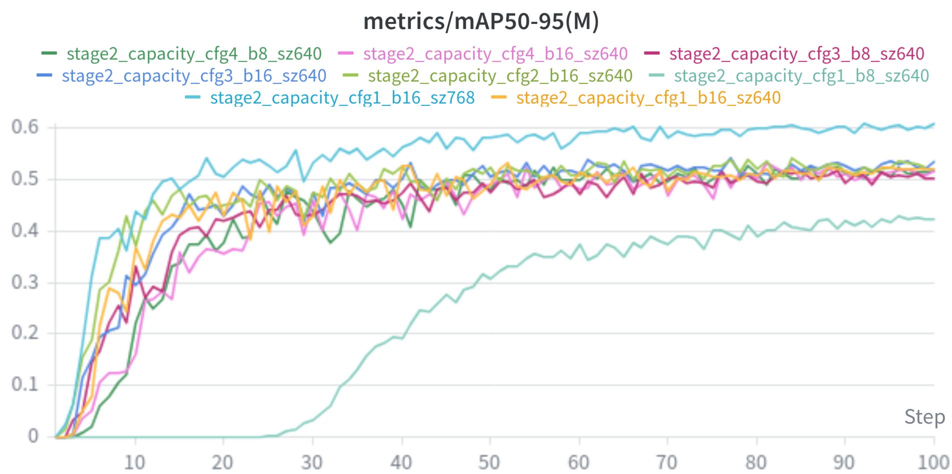

Figure 4.2: Validation mask $\text{mAP}_{50:95}$ for the successful unique Stage 2 capacity-tuning configurations. The best result was obtained with batch size 16 and image size 768, which was selected for Stage 3 refinement.

Table 4.1: Best validation results of the baseline and tuned YOLOv8-Seg models.

Model	Best epoch	Mask mAP_{50}	Mask $\text{mAP}_{50:95}$	Mask precision	Mask recall
Baseline YOLOv8-Seg	49	0.93931	0.59645	0.90623	0.89947
Stage 3 tuned YOLOv8-Seg	120	0.94268	0.63064	0.90150	0.90810

Table 4.2: Held-out test-set comparison between the baseline and the selected Stage 3 tuned YOLOv8-Seg model.

Model	Image size	Mask mAP_{50}	Mask $\text{mAP}_{50:95}$	Mask precision	Mask recall
Baseline YOLOv8-Seg	640	0.92819	0.73744	0.87603	0.90832
Stage 3 tuned YOLOv8-Seg	640	0.91706	0.68829	0.86839	0.88443
Baseline YOLOv8-Seg	768	0.92638	0.74752	0.86778	0.92329
Stage 3 tuned YOLOv8-Seg	768	0.92719	0.71985	0.87399	0.90724

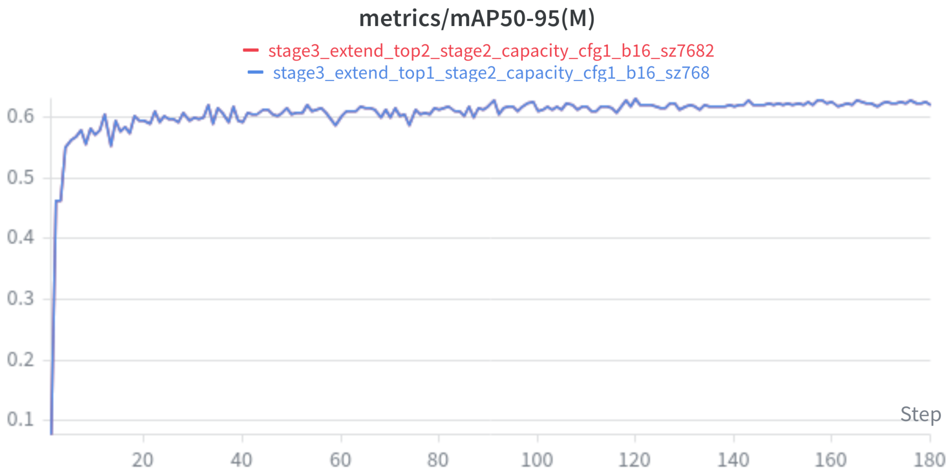

Figure 4.3: Stage 3 validation mask $mAP_{50:95}$ for the two refinement runs. The runs followed nearly identical trajectories and converged to the same performance region.

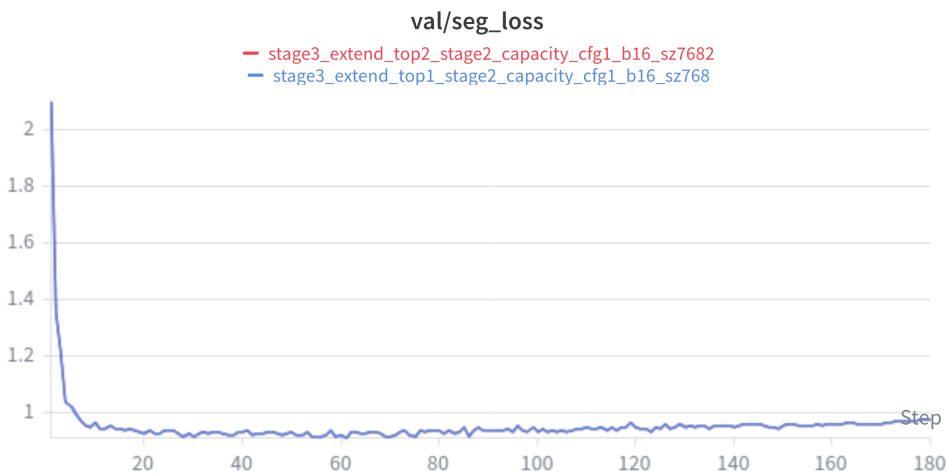

Figure 4.4: Stage 3 validation segmentation loss. After a steep initial decrease, the loss remained stable, indicating consistent convergence during refinement.

remained better on the final test set and was therefore retained as the reference model for later deployment-oriented experiments.

4.1.3 Comparison with YOLO26

YOLO26-small was evaluated under comparable training conditions to check whether the newer architecture could outperform the YOLOv8-Seg baseline. As shown in Table 4.3, it did not. The longer 180-epoch YOLO26 run improved its validation curve slightly (Figure 4.5), but the final test-set ranking remained in favor of the baseline model.

Table 4.3: Test-set comparison between the baseline YOLOv8-Seg model and YOLO26-small.

Model	Image size	Mask mAP_{50}	Mask $mAP_{50:95}$	Mask precision	Mask recall
YOLOv8-Seg baseline	640	0.91921	0.58295	0.86835	0.89896
YOLO26-small	640	0.91209	0.57538	0.87300	0.89879
YOLOv8-Seg baseline	768	0.92384	0.62325	0.86577	0.91939
YOLO26-small	768	0.92596	0.61689	0.88408	0.91326

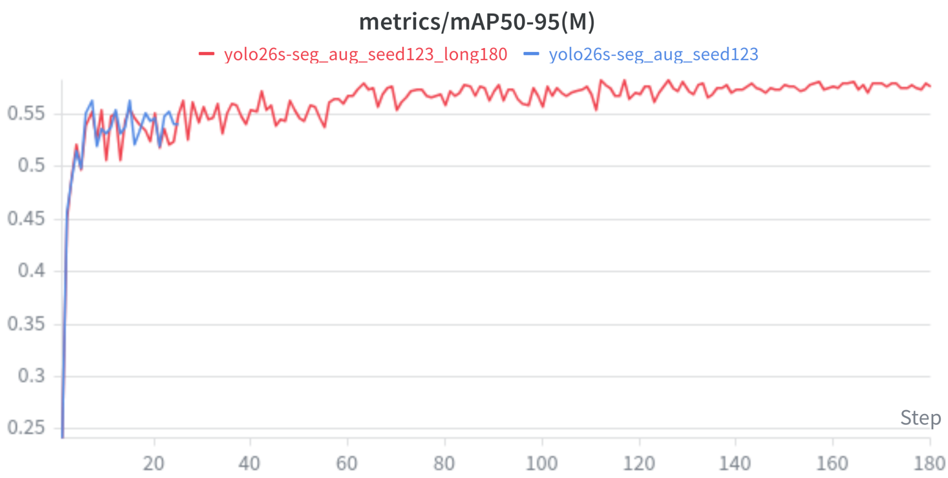

Figure 4.5: Validation mask $mAP_{50:95}$ for the two YOLO26 training runs. The longer 180-epoch run improved over the shorter run, but the resulting model still did not surpass the YOLOv8 baseline on the held-out test set.

4.1.4 Environment-specific fine-tuning with ADC data

The final YOLO experiment addressed domain adaptation to the target university environment. The baseline checkpoint was fine-tuned on the original dataset augmented with 46 newly collected and annotated images from HAN University, Building 31. Table 4.4 shows that this adaptation step did not materially degrade performance on the original held-out test split: mask $mAP_{50:95}$ remained effectively unchanged (0.58295 vs. 0.58293). The validation curve in Figure 4.6 is consistent with this observation.

The effect on the environment-specific test set is much stronger, as shown in Table 4.5. Mask $mAP_{50:95}$ increased from 0.12290 to 0.50463 after fine-tuning, while precision and recall

Table 4.4: Performance on the original held-out elevator test split before and after ADC fine-tuning.

Model	Mask mAP_{50}	Mask $mAP_{50:95}$	Mask precision	Mask recall
Baseline YOLOv8-Seg	0.91921	0.58295	0.86835	0.89896
ADC fine-tuned YOLOv8-Seg	0.91912	0.58293	0.88043	0.90050

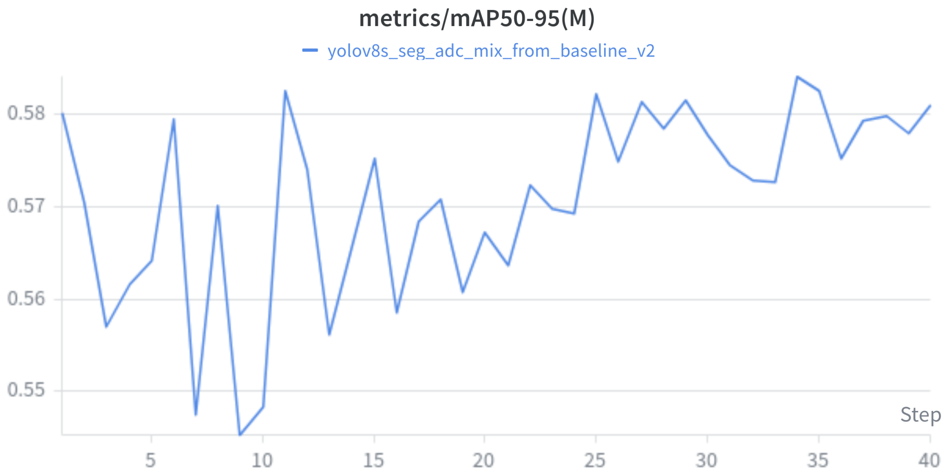

Figure 4.6: Validation mask $mAP_{50:95}$ during ADC-based fine-tuning. The curve remains close to the baseline validation range, which aligns with the near-identical performance observed on the original held-out test split.

also improved substantially. This is the most practically important YOLO result of the study: a relatively small amount of deployment-specific data produced a large gain in the target environment without sacrificing the original-domain test performance.

Table 4.5: Performance on the environment-specific ADC test split before and after ADC fine-tuning.

Model	Mask mAP_{50}	Mask $mAP_{50:95}$	Mask precision	Mask recall
Baseline YOLOv8-Seg	0.53520	0.12290	0.61209	0.70136
ADC fine-tuned YOLOv8-Seg	0.99500	0.50463	1.00000	0.99712

Overall, the YOLO results show three main findings. First, the baseline YOLOv8-Seg model was already strong and difficult to surpass on the general elevator test set. Second, staged hyperparameter tuning improved validation performance but did not fundamentally change the final model ranking. Third, the most practically valuable improvement was achieved through environment-specific fine-tuning with ADC data, which substantially increased performance on the target deployment domain while preserving performance on the original dataset.

4.2 OCR hyperparameter tuning and evaluation

The OCR experiments focused on improving cropped-button recognition while keeping the recognizer lightweight enough for later embedded deployment. As in the YOLO experiments, model selection during OCR tuning was based on the validation split, and the final comparison between OCR candidates was performed on the held-out OCR test set.

4.2.1 Stage 1: epoch selection

The first OCR tuning stage varied only the number of training epochs while keeping the optimizer configuration fixed. The main objective of this stage was to determine whether the original 60-epoch fine-tuning recipe was necessary or whether a shorter training budget would already achieve the same validation quality.

Table 4.6 shows that the 50-epoch and 60-epoch runs converged to almost identical validation performance, both reaching their best result at epoch 46. In contrast, the 30-epoch run underperformed clearly. This indicates that the recognizer required more than 30 epochs to converge, but that training beyond 50 epochs provided no meaningful additional benefit. For that reason, 50 epochs were selected as the fixed budget for the next stage of OCR hyperparameter tuning. This interpretation is also supported by the Stage 1 training-loss curves shown in Figure 4.7, where the main decrease occurs well before the end of the 50-epoch run and the longer schedule does not suggest a qualitatively different convergence regime.

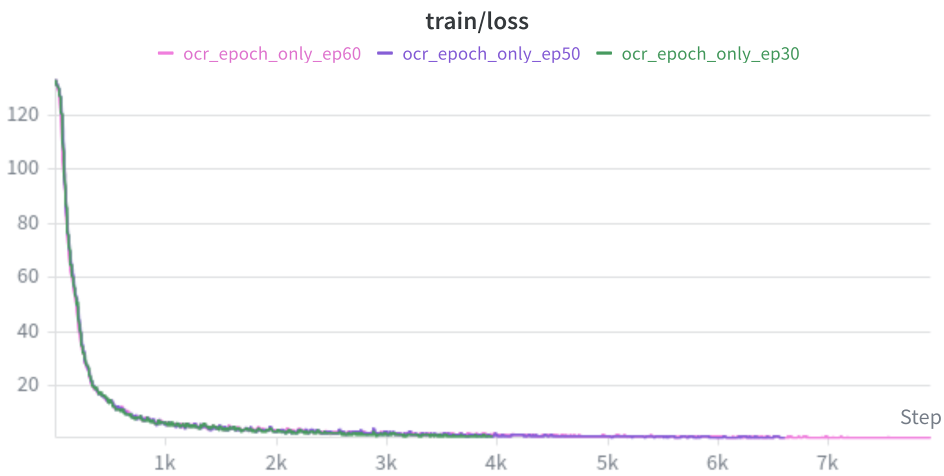

Figure 4.7: Stage 1 OCR training loss for the epoch-selection runs. Most of the optimization progress occurs before the 50-epoch mark, which supports the decision to use 50 epochs for the subsequent optimizer sweep.

Table 4.6: Stage 1 OCR epoch-sweep results on the validation split.

Trial	Best epoch	Accuracy	Normalized edit distance
30 epochs	16	0.78776	0.83732
50 epochs	46	0.84124	0.87080
60 epochs	46	0.84057	0.86893

4.2.2 Stage 2: learning rate and weight decay sweep

After fixing the training budget at 50 epochs, a second tuning stage evaluated a narrow optimizer sweep over learning rate and weight decay. Four configurations were tested. The resulting validation scores are summarized in Table 4.7.

The best configuration used an initial learning rate of 3×10^{-4} and a weight decay of 3×10^{-5}. This run achieved a validation accuracy of 0.84157 and a normalized edit distance of 0.87098, making it the strongest OCR candidate of the sweep. The configuration with the same learning rate but lower weight decay performed substantially worse, while the run using 5×10^{-4} degraded sharply and converged to a much lower performance level. Increasing the learning rate further to 7×10^{-4} recovered most of the performance, but still remained below the best 3×10^{-4} run.

These results indicate that OCR fine-tuning was sensitive to the optimizer settings even within a relatively narrow search range. In particular, the stage-2 results suggest that a slightly lower learning rate than the original baseline provided more stable and ultimately better convergence on the recognition task. This behavior is visible in Figure 4.8, where the best run maintains the highest evaluation accuracy, and in Figure 4.9, where the same configuration also produces the strongest normalized edit-distance curve.

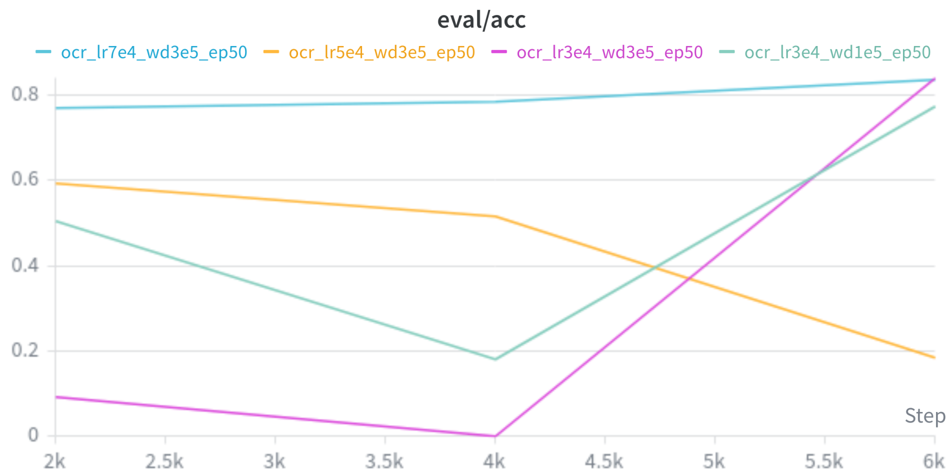

Figure 4.8: Stage 2 OCR evaluation accuracy for the learning-rate and weight-decay sweep. The configuration with learning rate 3×10^{-4} and weight decay 3×10^{-5} achieved the best validation accuracy.

Table 4.7: Stage 2 OCR hyperparameter sweep on the validation split.

Trial	Learning rate	Weight decay	Accuracy	Normalized edit distance
lr3e4_wd1e5_ep50	3×10^{-4}	1×10^{-5}	0.77531	0.83148
lr3e4_wd3e5_ep50	3×10^{-4}	3×10^{-5}	**0.84157**	**0.87098**
lr5e4_wd3e5_ep50	5×10^{-4}	3×10^{-5}	0.59603	0.74899
lr7e4_wd3e5_ep50	7×10^{-4}	3×10^{-5}	0.83653	0.86546

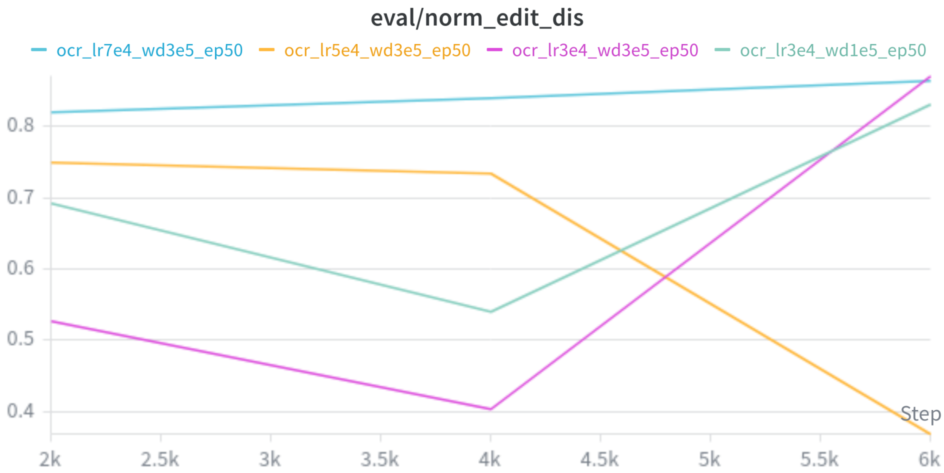

Figure 4.9: Stage 2 OCR normalized edit distance for the learning-rate and weight-decay sweep. The best-performing configuration also achieved the highest normalized edit-distance score.

4.2.3 Final OCR benchmark

The final OCR benchmark compared the original fine-tuned recognizer against the best-performing stage-2 configuration on the held-out OCR test set. The results are shown in Table 4.8. The stage-2 model outperformed the initial OCR model on both recognition-quality metrics. Test accuracy improved from 0.81635 to 0.83486, while normalized edit distance improved from 0.85574 to 0.87190. In addition, the measured evaluation throughput increased slightly from 805.68 to 817.71 samples per second.

Taken together, these results show that the OCR hyperparameter tuning process produced a real improvement rather than only a validation-specific gain. Unlike the YOLO tuning experiments, where better validation performance did not translate into a better final test-set model, the OCR stage-2 winner also improved the held-out test benchmark. For this reason, the stage-2 best OCR model was selected as the final recognition candidate for later integration into the complete perception pipeline.

Table 4.8: Held-out OCR test-set comparison between the initial recognizer and the selected stage-2 OCR model.

Model	Accuracy	Normalized edit distance	Throughput (samples/s)
Initial OCR model	0.81635	0.85574	805.68
Stage-2 best OCR model	**0.83486**	**0.87190**	**817.71**

4.2.4 Model hardware optimization results

As described in Chapter 3, the final YOLO and OCR models were exported to TensorRT for optimized execution on the NVIDIA Jetson platform. To verify the effect of this export step, a compact benchmark was performed on the held-out test image `367.jpg`. For YOLO, the comparison used the PyTorch checkpoint and the exported TensorRT engine on the same full image. For OCR, the comparison used the Paddle inference model and the exported TensorRT engine on the five annotated button crops from the same image.

Table 4.9: Single-image model hardware optimization benchmark on test image `367.jpg`. For OCR, the reported latency is average time per single button crop.

Model stage	Baseline backend	TensorRT backend	Speedup
YOLO inference [ms/image]	36.46	24.43	1.49×
YOLO wall time [ms/image]	64.19	64.52	1.00×
OCR latency [ms/crop]	167.72	10.29	16.30×

The YOLO result shows that TensorRT reduced the forward-pass time substantially, even though the total offline single-image wall time remained almost unchanged because preprocessing and postprocessing still contributed a large fraction of the runtime. The OCR result is more direct: the TensorRT engine reduced average latency from 167.72 ms to 10.29 ms per button crop while preserving the same accepted predictions on all five button regions in the test image. These results support the use of TensorRT export as an important deployment optimization step before integrating the models into the full ROS 2 pipeline.

4.3 ROS 2 pipeline performance

After the final YOLO and OCR models were exported to TensorRT, the complete perception stack was evaluated on the NVIDIA Jetson Orin Nano in its deployed ROS 2 configuration. The purpose of these experiments was not to compare neural-network accuracy, but to quantify the practical throughput of the integrated pipeline under different runtime settings. The reported measurements correspond to the final post-refactor pipeline in which ROI-local masks are published by the YOLO node and full-image annotation reconstruction is performed only inside the ADC when export is required. During these experiments, the Stereolabs ZED X camera was launched with the `zedx` wrapper profile, which uses a native `HD1200` grab mode at 30 Hz and publishes lens-corrected color and depth data at 15 Frames Per Second (FPS). With the configured downscale factor of 2.0, the effective published image resolution used by the perception pipeline was 960×600 pixels.

Three final runtime scenarios were compared:

- camera + YOLO + OCR + ADC,
- camera + YOLO + OCR without collector and with OCR uncapped, and
- camera + YOLO + OCR without collector and with OCR limited to four regions per message.

Table 4.10 summarizes the resulting topic throughput. The collector-enabled configuration sustained approximately 10 FPS while preserving the full deployment data-collection path. When the collector was disabled and OCR was allowed to process all candidate regions, the end-to-end rate increased to about 12.2 FPS. The best runtime-oriented configuration was obtained when OCR was capped to four regions per message, yielding approximately 13.1 FPS end-to-end on the final `/button_targets_3d_ocr` output topic. The reported topic rates were measured with the standard `ros2 topic hz` utility on the relevant output topics.

Table 4.10: Final ROS 2 pipeline throughput on the NVIDIA Jetson Orin Nano after the ROI-mask refactor, measured with `ros2 topic hz`.

Configuration	`/button_targets_3d` FPS	`/button_targets_3d_ocr` FPS
With collector	~ 10.0	~ 10.0
No collector, OCR uncapped	~ 12.15	~ 12.29
No collector, OCR capped to 4 ROIs	~ 13.14–13.47	~ 13.08–13.25

The timing breakdown confirms that the final system bottleneck is not the TensorRT forward pass alone, but the combined cost of segmentation postprocessing, depth estimation, and OCR execution inside the live ROS 2 graph. Table 4.11 shows typical timing ranges of the YOLO and OCR nodes for the three runtime configurations. In collector mode, the YOLO node required approximately 81–91 ms per callback and the OCR node 79–95 ms. Removing the collector while keeping OCR uncapped reduced the practical throughput penalty of data export but still required roughly 11 processed OCR regions per message, which kept OCR total time in the 77–102 ms range. Limiting OCR to four regions reduced OCR total time to roughly 33–35 ms and allowed the complete pipeline to reach its best measured throughput.

Table 4.11: Typical node timing ranges for the final ROS 2 runtime experiments.

Configuration	YOLO total [ms]	YOLO depth [ms]	OCR total [ms]	OCR processed ROIs
With collector	~ 81–91	~ 3.6–6.6	~ 79–95	not capped
No collector, OCR uncapped	~ 78–99	~ 4–7	~ 77–102	~ 10.7–10.9
No collector, OCR capped to 4 ROIs	~ 70.4–77.8	~ 5.3–7.0	~ 33.0–35.2	4.0

These results show that the integrated perception pipeline can operate at deployment-relevant frame rates on the NVIDIA Jetson Orin Nano, provided that annotation serialization is kept out of the real-time YOLO path and OCR workload is bounded. For runtime operation, the most effective configuration is the ROI-limited OCR setup without concurrent collection, which achieved approximately 13.1 FPS. For deployment sessions that must simultaneously gather new training data, the collector-enabled configuration remains practical at approximately 10 FPS, which is sufficient to preserve the full adaptive data-collection loop described in Chapter 3.

5 Discussion

This chapter interprets the results presented in Chapter 4 and relates them to the project objective of developing an efficient and scalable computer-vision workflow for robot interaction with elevator infrastructure on embedded hardware.

5.1 Interpretation of the main findings

The main outcome of this project is that a practical embedded perception workflow can be achieved by combining instance-segmentation-based button detection, crop-based OCR, and a modular ROS 2 runtime pipeline. The results show that the strongest overall system did not come from selecting the newest model architecture or from maximizing validation performance in isolation. Instead, the most effective workflow emerged from a sequence of engineering decisions that balanced perception quality, deployment constraints, and future scalability.

For YOLO, the baseline YOLOv8-Seg model remained the most reliable choice for deployment. Hyperparameter tuning improved validation performance, but those gains did not translate into a better final held-out test model. Likewise, YOLO26-small did not outperform the baseline under the same evaluation conditions. This is an important result because it shows that, for this task, a carefully chosen and well-understood baseline can be more valuable than adopting a newer architecture without a clear deployment benefit. The most practically useful YOLO improvement came from environment-specific fine-tuning with ADC data from HAN University, Building 31, which substantially improved performance in the target environment while preserving the original-domain performance.

For OCR, the project produced a clearer tuning outcome. The selected stage-2 configuration improved both test accuracy and normalized edit distance relative to the initial recognizer. In contrast to the YOLO tuning results, the OCR validation improvements did translate into better held-out test-set performance. This suggests that the OCR stage was more sensitive to optimization settings in a way that remained meaningful at test time.

At system level, the deployed ROS 2 pipeline demonstrates that model quality alone is not sufficient for real-time embedded performance. The final pipeline throughput depended strongly on how much non-inference work remained in the runtime path. The shift from full-image mask export to compact ROI-local mask handling, together with the split-node YOLO/OCR architecture and the bounded OCR workload, produced the most important runtime gains. The best measured configuration achieved about 13.1 FPS, while the collector-enabled configuration remained practical at about 10 FPS. This shows that the embedded performance target was not reached by model export alone, but by a combination of model export, message-design decisions, and workload placement.

5.2 Relation to the research questions

The results provide a coherent answer to the main research question. An efficient and scalable computer-vision workflow for hospital interaction-point perception can be designed by using instance segmentation for button localization, crop-based OCR for semantic interpretation, TensorRT export for embedded execution, and an ROS 2-based split-node architecture for modular runtime integration.

The first sub-question concerned the relevant human-interaction points in hospital environments. The project focused on elevator buttons and related panel interfaces because they are a concrete and operationally critical interaction class for multi-floor service-robot mobility. This focus proved appropriate: the task is sufficiently challenging to expose problems of detection, segmentation, recognition, and embedded deployment, while still being narrow enough to support a complete workflow.

The second sub-question asked which methods and architectures were suitable. The literature review suggested that instance segmentation is better aligned with precise robot interaction than coarse object detection alone, and the results support this choice. The segmentation masks were directly useful for masked depth estimation and 3D target generation. The final architecture therefore combined a segmentation-based detector with a separate recognizer rather than relying on a single end-to-end model.

The third sub-question concerned dataset design. The results show that dataset quality and dataset relevance were both important. The original curated dataset was sufficient to train a strong baseline, but the environment-specific fine-tuning experiment demonstrated that even a relatively small amount of deployment-collected data can substantially improve performance in the target domain. This supports the decision to include an ADC in the full workflow.

The fourth sub-question asked which optimization techniques support embedded inference. Here the answer is twofold. First, TensorRT export was beneficial for both YOLO and OCR, with especially strong gains for OCR. Second, export alone was not enough: runtime-side optimizations such as reducing annotation overhead, separating the inference nodes, and limiting the number of processed OCR regions were equally important.

The fifth sub-question concerned scalability and efficient future improvement. The final system addresses this directly through the ADC, which turns runtime use into a source of curated retraining data. The environment-specific YOLO improvement demonstrates that this design is not only theoretically scalable, but also practically useful.

5.3 Limitations

Several limitations should be considered when interpreting the results. First, the project focused on elevator-interface perception rather than the full set of hospital interaction elements. While this was a justified and useful scope reduction, the final conclusions should not automatically be generalized to all door panels, access readers, or human-machine interfaces.

Second, some benchmark results remain sensitive to the evaluation setup. For example, the single-image framework-to-TensorRT comparison is useful for illustrating acceleration trends, but it is not by itself a complete deployment benchmark. Likewise, some distance-related YOLO improvements were most clearly observed in field behavior and specific replay analyses rather than in a single universal summary metric. For this reason, the confidence and coverage improvements are stronger conclusions than any claim of a general doubling of recognition range.

Third, the project was executed on a single embedded platform and camera setup. The

NVIDIA Jetson Orin Nano and Stereolabs ZED X are representative for the target use case, but performance and tuning decisions may differ on other edge devices or other sensing configurations.

Fourth, the report currently emphasizes perception and runtime integration rather than downstream manipulation or actuation. The system produces 3D button targets and button labels, but the full closed-loop robot interaction behavior lies beyond the present project scope.

5.4 Implications for future robot deployment

Despite these limitations, the project has clear practical implications. The results suggest that a service robot does not require a monolithic perception stack to achieve useful embedded performance. A modular pipeline with explicit message interfaces can be easier to tune, benchmark, and extend. This is particularly important for educational and research robots such as CareBot, where iterative development and subsystem replacement are expected.

The project also suggests that deployment data should be treated as a core part of the system design rather than as an afterthought. The inclusion of the ADC makes the perception workflow self-improving in a structured way: the same runtime system used in deployment can collect higher-value examples for the next training cycle. This closes an important loop between experimentation, deployment, and future model improvement.

Overall, the project shows that the combination of instance segmentation, task-specific OCR, TensorRT acceleration, and structured deployment data collection provides a credible foundation for later integration into a more complete autonomous hospital robot.

6 Conclusions and recommendations

6.1 Conclusions

The objective of this project was to develop and validate an efficient and scalable computer-vision workflow that enables a service-robot platform to detect and classify human-interaction elements in hospital environments using embedded hardware. Based on the results presented in this report, this objective was achieved for the targeted elevator-interaction use case.

The project demonstrates that a practical embedded perception workflow can be constructed from four main components: a strong instance-segmentation baseline for button localization, a crop-based OCR stage for button interpretation, TensorRT export for hardware-efficient inference, and a modular ROS 2 pipeline that integrates detection, 3D projection, text recognition, and deployment data collection.

The main conclusions are as follows.

1. Instance segmentation is an effective method for elevator-button perception because it supports both accurate 2D localization and mask-based depth estimation for 3D target generation.

2. The baseline YOLOv8-Seg model was the strongest deployment candidate among the evaluated detector variants. Hyperparameter tuning improved validation performance, but not the final held-out test ranking.

3. Environment-specific fine-tuning with ADC data from HAN University, Building 31 produced the most practically valuable YOLO improvement, substantially increasing target-domain performance while preserving original-domain performance.

4. The selected stage-2 OCR model improved held-out recognition quality and formed a suitable semantic interpretation stage for the full pipeline.

5. TensorRT export was beneficial for both deployed models. The effect was moderate but clear for YOLO inference time, and very strong for OCR latency.

6. The final ROS 2 pipeline achieved practical embedded runtime performance on the NVIDIA Jetson Orin Nano, with the best measured configuration operating at about 13.1 FPS and the collector-enabled configuration remaining usable at about 10 FPS.

7. Scalability is supported not only by model architecture choices, but also by the inclusion of the ADC, which enables structured deployment-time data collection for future retraining.

Taken together, these conclusions answer the main research question: an efficient and scalable computer-vision workflow for hospital interaction-point perception can be achieved on embedded hardware by combining segmentation-based detection, task-specific recognition, deployment-oriented optimization, and iterative deployment data collection within a modular ROS 2 architecture.

6.2 Recommendations

The following recommendations are proposed for future work and further development of the system.

1. Extend the perception scope beyond elevator buttons to additional hospital interaction elements such as access readers, door controls, and other panel interfaces.

2. Perform larger and more controlled deployment trials in multiple buildings so that domain shift, lighting variation, and panel diversity can be evaluated more systematically.

3. Continue using the ADC as part of regular deployment, with periodic review and retraining cycles based on newly collected samples.

4. Investigate whether task-specific filtering or ranking strategies can further reduce unnecessary OCR workload without harming end-to-end recognition quality.

5. Evaluate the pipeline on additional embedded platforms to determine how transferable the optimization choices are beyond the current NVIDIA Jetson-based setup.

6. Integrate the perception pipeline with downstream robot action modules so that the 3D target estimates can be validated in full interaction tasks rather than perception-only experiments.

7. Refine the final report by moving low-value implementation detail out of the main chapters when necessary, while keeping the central research narrative focused on the validated workflow and its deployment implications.

6.3 Final reflection

This project provides a concrete foundation for autonomous interaction with elevator infrastructure in hospital-like environments. More broadly, it shows that robust embedded perception is not achieved by model selection alone. The best results emerged from treating dataset design, model choice, runtime architecture, and deployment data collection as one connected system. That systems perspective is likely to remain essential as the CareBot platform evolves toward more complete autonomous operation in real healthcare environments.

Bibliography

[1] Bai, J., Lu, F., Zhang, K., et al. (2019). Onnx: Open neural network exchange. In *Proceedings of the 2019 ACM International Conference on Multimedia Systems*, pages 460–466.

[2] Bernard, A., Hörnle, T., and Dillmann, R. (2020). Vision-based perception of control interfaces for robotic manipulation. *Robotics and Autonomous Systems*, 124:103386.

[3] Chen, T., Chen, Z., and Chen, Y. (2018). Deep learning in robotics: A review of recent research. *Advanced Robotics*, 32(5):201–214.

[4] Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., and Schiele, B. (2016). The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3213–3223.

[5] He, K., Gkioxari, G., Dollár, P., and Girshick, R. (2017). Mask r-cnn. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2961–2969.

[6] Holland, J. et al. (2021). Service robots in the healthcare sector. *Robotics*, 10(1):47.

[7] Iguazio (2020). Nuclio: High-performance serverless event and data processing platform. `https://nuclio.io`.

[8] Jocher, G., Chaurasia, A., and Qiu, J. (2023). Yolov8: Ultralytics next-generation yolo models. *arXiv preprint arXiv:2305.09972*.

[9] Kang, Y., Kim, H., and Lee, J. (2020). Edge ai: On-device intelligence for embedded systems. *IEEE Consumer Electronics Magazine*, 9(4):20–27.

[10] Lin, T.-Y., Maire, M., Belongie, S., et al. (2014). Microsoft coco: Common objects in context. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 740–755.

[11] Liu, J., Fang, Y., Zhu, D., Ma, N., Pan, J., and Meng, M. Q.-H. (2021). A large-scale dataset for benchmarking elevator button segmentation and character recognition. *IEEE Transactions on Industrial Informatics.*

[12] Long, J., Shelhamer, E., and Darrell, T. (2015). Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3431–3440.

[13] Martínez, M., Montero, R., and Pérez, E. (2018). Irso: A dataset for interactive robotic service operations in indoor environments. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1234–1241.

[14] Masala, G. L. et al. (2025). Artificial intelligence and assistive robotics in healthcare: A narrative review. *International Journal of Environmental Research and Public Health*, 22(5):781.

[15] Merkel, D. (2014). Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239).

[16] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788.

[17] Ren, S., He, K., Girshick, R., and Sun, J. (2017). Faster r-cnn: Towards real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6):1137–1149.

[18] Sekachev, B., Manovich, N., Zhiltsov, M., et al. (2020). Cvat: Computer vision annotation tool. *arXiv preprint arXiv:2009.13245*.

[19] Sun, X., Wu, J., Zhang, X., Zhang, Z., Zhang, C., Xue, T., Freeman, W., and Tenenbaum, J. (2017). Scenenet rgb-d: Can 5m synthetic images beat generic imagenet pre-training? In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2678–2687.

[20] Tan, M. and Le, Q. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 6105–6114.

[21] Wang, X., Luo, C., and Zhang, Y. (2019). Vision-based perception for autonomous indoor service robots: A survey. *IEEE Transactions on Automation Science and Engineering*, 16(4):1603–1619.

A Supplementary model evaluations

A.1 Mask R-CNN checkpoint progression during annotation support

The Mask R-CNN model was used as an annotation-support model within the CVAT–Nuclio workflow, rather than as the final deployment model. During dataset construction, the model was retrained several times as more annotated images became available. To quantify the effect of this iterative retraining, four checkpoints were evaluated on the same held-out test split used for the annotation dataset. Table A.1 reports both bounding-box and mask metrics, with mask AP treated as the primary segmentation indicator.

Table A.1: Held-out test-set performance of the annotation-support Mask R-CNN checkpoints as the training set grew during the annotation process.

Checkpoint stage	Training images	BBox AP@[.50:.95]	Mask AP@[.50:.95]	Mask AP@0.50
Initial checkpoint	90	0.5878	0.5844	0.7776
Retrained checkpoint	449	0.6682	0.6660	0.8284
Retrained checkpoint	900	0.7007	0.6903	0.8772
Final checkpoint	1400	0.7192	0.7000	0.8990

The results show a consistent improvement trend as the amount of annotated data increased. The largest gain was observed between the initial 90-image checkpoint and the first larger retrained checkpoint at 449 images, while later retraining to 900 and 1400 images provided smaller but still measurable improvements. On the primary mask metric, AP@[0.50:0.95] increased from 0.5844 to 0.7000. This supports the annotation strategy adopted in this project: iterative retraining of the pre-annotation model improved the quality of automatically proposed instances, thereby reducing the manual correction burden in later annotation rounds.

B ROS 2 pipeline parameter reference

This appendix summarizes the main runtime parameters of the deployed ROS 2 perception pipeline. The tables are intended as a practical reference for reproducing experiments or adjusting node behavior during deployment. Values shown in the "Default / configured value" column correspond to the final thesis configuration where this was defined explicitly in the YAML files; otherwise the node's declared software default is reported.

B.1 YOLO inference node

Table B.1: Main parameters of `yolo_engine_node`.

Parameter	**Default / configured value**	**Purpose**
`engine_path`	launch-time path	Path to the TensorRT YOLO engine.
`image_topic`	`/zed/zed_node/rgb/color/rect/image`	Input image topic.
`depth_topic`	`/zed/zed_node/depth/depth_registered`	Input depth topic used for 3D projection.
`camera_info_topic`	`/zed/zed_node/rgb/color/rect/camera_info`	Camera intrinsics topic.
`detections_topic`	`/detections`	Optional 2D detection topic retained for debugging.
`targets_3d_topic`	`/button_targets_3d`	Main 3D target output topic.
`conf`	0.50	Confidence threshold for accepted detections.
`infer_hz`	8.0	Timer frequency for inference callbacks.
`enable_distance_estimation`	`true`	Enables depth-based 3D target estimation.
`max_depth_age_ms`	150.0	Maximum allowed age of buffered depth data.
`min_depth_m`	0.10	Minimum accepted depth value.
`max_depth_m`	10.0	Maximum accepted depth value.
`min_valid_depth_pixels`	20	Minimum number of valid masked depth pixels.
`depth_percentile`	50.0	Percentile used for representative target depth.
`enable_ocr`	`false`	Legacy inline OCR mode; disabled in the split-node pipeline.
`ocr_engine_path`	empty	Optional legacy inline OCR engine path.
`ocr_input_name`	`x`	Legacy inline OCR input tensor name.
`ocr_charset_path`	empty	Optional legacy inline OCR character set path.
`ocr_input_h`	48	Legacy inline OCR input height.
`ocr_input_w`	320	Legacy inline OCR input width.
`ocr_min_conf`	0.60	Legacy inline OCR confidence threshold.
`ocr_max_rois_per_frame`	8	Legacy inline OCR ROI cap.
`warmup_runs`	3	Number of startup warmup inferences.
`enable_mask_rle_export`	`false`	Enables direct mask export for dataset collection.
`publish_debug_image`	`true`	Enables publication of a visual debug image.
`debug_image_topic`	`/yolo/debug_image`	Debug image output topic.
`enable_perf_logging`	`true`	Enables periodic runtime timing logs.

Parameter	Default / configured value	Purpose
perf_log_every_n	50	Logging interval for performance statistics.

B.2 OCR inference node

Table B.2: Main parameters of ocr_trt_node.

Parameter	Default / configured value	Purpose
image_topic	/zed/zed_node/rgb/color/rect/image	Input topic used for buffered crops.
targets_topic_in	/button_targets_3d	Input topic from the YOLO node.
targets_topic_out	/button_targets_3d_ocr	Output topic with text-enriched targets.
debug_image_topic	/ocr/debug_image	Debug visualization topic.
ocr_engine_path	launch-time path	Path to the TensorRT OCR engine.
ocr_input_name	x	Name of the input tensor in the engine.
ocr_charset_path	empty	Optional character set file path.
ocr_input_h	48	Input crop height for recognition.
ocr_input_w	320	Input crop width for recognition.
ocr_min_conf	0.60	Minimum accepted text confidence.
ocr_max_rois_per_msg	8	Maximum number of button crops processed per message.
warmup_runs	3	Number of startup warmup inferences.
max_image_age_ms	220.0	Maximum age of buffered image data matched to a target message.
image_buffer_size	30	Number of recent images retained for matching.
publish_debug_image	declared default true	Enables publication of the OCR debug image.
enable_perf_logging	declared default true	Enables periodic timing logs.
perf_log_every_n	declared default 50	Logging interval for performance statistics.
enable_fuzzy_map	true	Enables post-recognition vocabulary correction.
label_vocab_path	configured file path	Path to known elevator button labels.
fuzzy_max_dist	1	Maximum edit distance for fuzzy label correction.
fuzzy_min_len	3	Minimum string length before fuzzy matching is applied.
fuzzy_min_score	0.0	Minimum score required before fuzzy correction is considered.

B.3 ADC

Table B.3: Main parameters of `active_data_collector`.

Parameter	**Default / configured value**	**Purpose**
`image_topic`	`/zed/zed_node/rgb/color/rect/image`	Input image topic.
`targets_topic`	`/button_targets_3d_ocr`	Input target topic from the OCR node.
`dataset_root`	configured dataset path	Root folder for collected sessions.
`session_name`	empty	Optional explicit session name.
`save_overlay_image`	`true`	Saves an annotated overlay image for each accepted sample.
`overlay_font_scale`	0.30	Font size used in overlay rendering.
`overlay_font_thickness`	1	Font thickness used in overlay rendering.
`max_image_age_ms`	250.0	Maximum age of the buffered image matched to a target message.
`image_buffer_size`	40	Number of recent images retained for timestamp matching.
`enable_perf_logging`	declared default `true`	Enables periodic collector timing logs.
`perf_log_every_n`	declared default 20	Logging interval for collector performance statistics.
`min_targets_per_frame`	1	Minimum number of accepted targets required for export.
`min_target_score`	0.45	Minimum detector score for a target candidate.
`require_text`	`True`	Requires accepted OCR output before export.
`min_text_score`	0.45	Minimum accepted OCR confidence.
`min_export_interval_sec`	0.75	Minimum time gap between accepted exports.
`min_bbox_area_pixels`	100.0	Minimum target bounding-box area.
`min_bbox_short_side_pixels`	10.0	Minimum short side of the target box.
`min_image_brightness`	35.0	Lower image brightness gate.
`max_image_brightness`	235.0	Upper image brightness gate.
`max_saturated_ratio`	0.25	Maximum saturated-pixel ratio.
`enable_sharpness_gate`	`true`	Enables whole-image sharpness filtering.
`min_laplacian_var`	120.0	Minimum whole-image Laplacian variance.
`enable_roi_sharpness_gate`	`true`	Enables target-ROI sharpness filtering.
`min_roi_laplacian_var`	40.0	Minimum ROI Laplacian variance.
`enable_stability_gate`	`true`	Enables temporal stability filtering across frames.
`stability_required_frames`	3	Number of supporting frames required for export.
`stability_iou_threshold`	0.40	IoU threshold for matching targets across frames.
`stability_depth_tolerance_m`	0.20	Depth consistency threshold in meters.
`stability_track_ttl_sec`	1.2	Lifetime of a temporary stability track.
`stability_require_ocr_consistency`	`true`	Requires stable OCR content across the track.
`enable_dedup`	`true`	Enables duplicate-frame rejection.
`dedup_hamming_threshold`	8	Hamming threshold for duplicate detection.

Parameter	Default / configured value	Purpose
`dedup_history_size`	50	Number of recent accepted samples used for deduplication.
`analysis_ocr_score_threshold`	0.60	Threshold used for analysis-only OCR statistics.
`distance_bin_size_m`	0.25	Bin size for distance-distribution statistics.
`max_analysis_distance_m`	5.0	Maximum distance included in exported analysis summaries.